

Short Technical Exercise (30 mins)

A short exercise to complete before your interview. It is deliberately small. We are not looking for a polished product; we want to see a working piece of code that you can explain, with tests, and a note on how you approached it.

Please keep it to around 30 minutes. If it is taking much longer, stop and send what you have with a note on what you would have done next.

The exercise: partner eligibility import

Perci works with B2B partners (insurers and employers) who send us lists of the people eligible for our service. Build a small TypeScript / Node.js program that imports one of these lists. Your program should:

- Read a CSV of eligible members with the columns `partner_member_id`, `first_name`, `last_name`, `date_of_birth`, `email`, `policy_start`, `policy_end`. Create your own sample file with a handful of rows, including a few that are deliberately broken.
- Store the valid rows somewhere simple. In-memory, a JSON file or SQLite are all fine.
- Be safe to run more than once. Importing the same file twice should not create duplicates. If a row has changed since the last import, the stored record should be updated.
- Reject rows that are invalid, and report what was rejected and why.
- Let us look up a member by `partner_member_id`, either through a small HTTP endpoint or a command-line call, whichever you prefer.
- Include automated tests for the behaviour you think matters most.

Some things are deliberately not specified, for example what counts as the same member if the email changes, or how strictly to validate dates. Make a decision, state it in your README, and move on. We are far more interested in how you handle the gaps than in whether you guess our preference.

What to send

- A link to a Git repository (public, or private with access granted to the address we give you).
- A README covering: how to run it and the tests, the assumptions you made, how you used AI tools while building it (we expect that you did, and we would like to know where they helped and where they did not), and how you checked that the result was correct.
- A short screen recording, two to three minutes, of you walking through the code and running the tests. Loom or similar is fine. We use this to hear you talk through your own work; it does not need to be tidy.

How we use it

Luke reviews it before your interview and it comes up briefly in the technical stage. It is a filter, not a score: we check that it runs, that it does what was asked, that the tests are meaningful, and that you can explain your decisions. There is no single right answer.

Questions

If anything is unclear, reply to the email this came with and we will help. There is no penalty for asking.